9.2.3.4.1 The hi/lo algorithm

The hi/lo algorithm splits the sequences domain into *hi* groups. A *hi* value is assigned synchronously, and every *hi* group is given a maximum number of *lo* entries, that can by assigned off-line without worrying about identifier value conflicts.

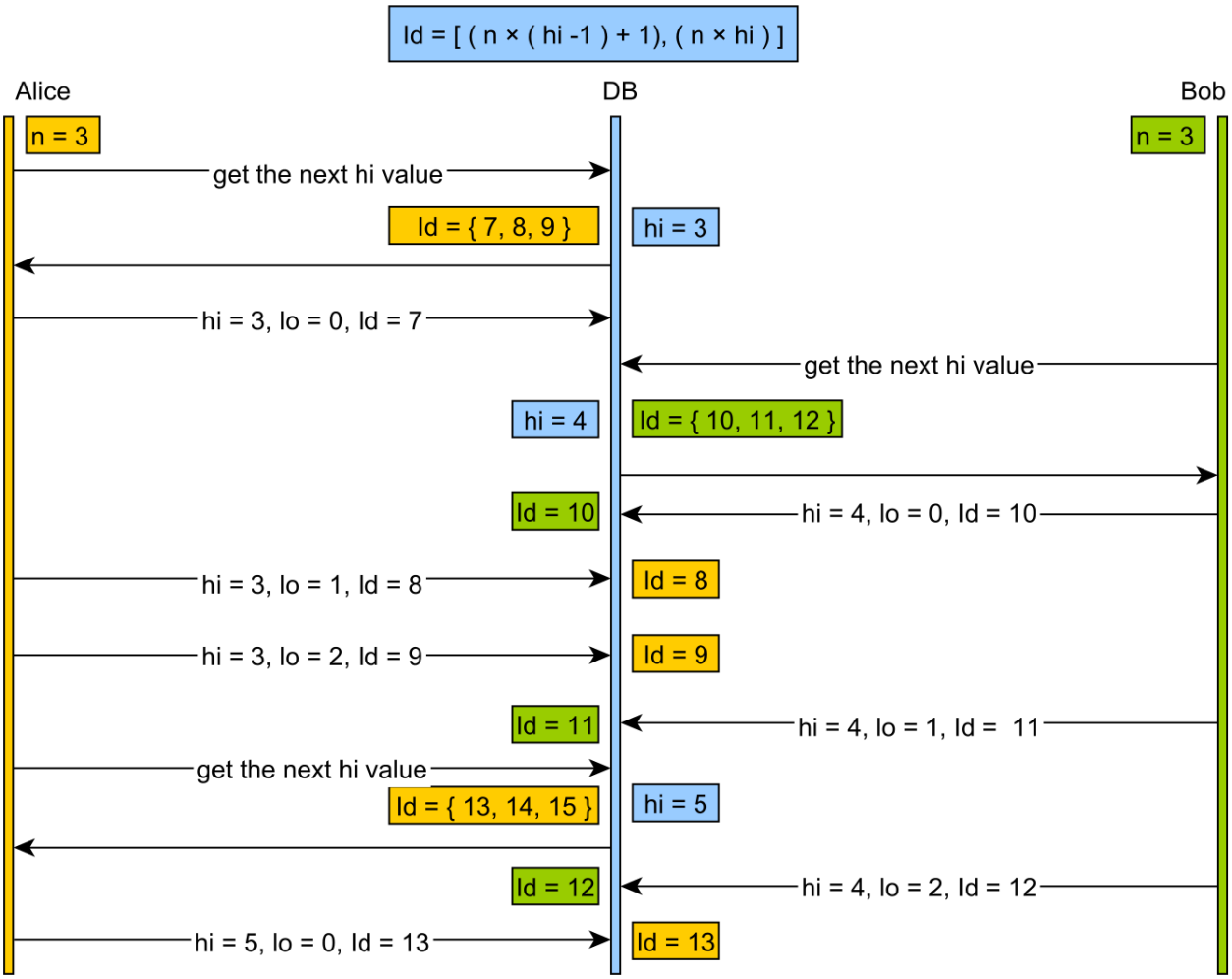

Figure 9.2: The hi/lo algorithm

1. The *hi* token is assigned either by the database sequence or the table generator, so two consecutive calls are guaranteed to see monotonically increasing values.
2. Once a *hi* token is retrieved, the *increment size* (*n - the number of *lo* entries*) defines the range of identifier values a transaction can safely allocate. The identifiers range is bounded by the following interval: $Id = [n \times (hi - 1) + 1, n \times hi]$ and the allocation is done as follows:
   - the current group values start from $n \times (hi - 1) + 1$
   - the *lo* value is taken from the following interval: {$$} [0, n - 1] {$$}
   - by adding the *lo* value to the initial group value, a unique identifier is obtained.
3. When all *lo* values are used, a new *hi* value is fetched, and the cycle continues.

The following example shows how the hi/lo algorithm works in practice. The entity is mapped as follows:

```
@Entity
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "hilo")
    @GenericGenerator(
        name = "hilo",
        strategy = "org.hibernate.id.enhanced.SequenceStyleGenerator",
        parameters = {
            @Parameter(name = "sequence_name", value = "sequence"),
            @Parameter(name = "initial_value", value = "1"),
            @Parameter(name = "increment_size", value = "3"),
            @Parameter(name = "optimizer", value = "hilo")
        }
    )
    private Long id;
}
```

Because the increment size is 3, the following test inserts 4 entities to show the number of database sequence calls.

```
doInJPA(entityManager -> {
    for(int i = 0; i < 4; i++) {
        Post post = new Post();
        entityManager.persist(post);
    }
});
```

Running the previous test generates the following output:

```
CALL NEXT VALUE FOR hilo_seqeunce
CALL NEXT VALUE FOR hilo_seqeunce

INSERT INTO post (id) VALUES (1)
INSERT INTO post (id) VALUES (2)
INSERT INTO post (id) VALUES (3)
INSERT INTO post (id) VALUES (4)
```

The first sequence call is for the first three values, while the second one is generated when reaching the forth entity that needs to be persisted. The more inserts a transaction requires, the better the performance gain from reducing the number of database sequence calls.

Unfortunately, this optimizer has a major limitation. Because the database sequence only assigns group values, all database clients must be aware of this algorithm. If the DBA must insert a row in the table above, he must use the hi/lo algorithm to determine the range of values that she can safely use.

For this reason, Hibernate offers other optimizer algorithms that are interoperable with external clients, unaware of the application-level optimization technique in use.

9.2.3.4.2 The default sequence identifier generator

The JPA identifier generator strategy only specifies the identifier type and not the algorithm used for generating such identifiers.

For the sequence generator, considering the following JPA mapping:

```
@Id
@GeneratedValue(generator = "sequence", strategy=GenerationType.SEQUENCE)
@SequenceGenerator(name = "sequence", allocationSize = 3)
private Long id;
```

Hibernate chooses the `SequenceHiLoGenerator` when the `hibernate.id.new_generator_mappings` configuration property is false. This was the default setting for Hibernate 3 and 4. The legacy `SequenceHiLoGenerator` uses the hi/lo algorithm, and, if the allocation size is greater than one, database interoperability could be compromised (every insert must be done according to the hi/lo algorithm rules).

If the aforementioned configuration property is true (the default setting for Hibernate 5), then the JPA mapping above will use the `SequenceStyleGenerator` instead.

Unlike its previous predecessor, the `SequenceStyleGenerator` uses configurable identifier optimizer strategies, and the application developer can even supply its own optimization implementation.

9.2.3.4.3 The default table identifier generator

Just like with sequences, the JPA table generator mapping can use a legacy or an enhanced generator, depending on the current Hibernate configuration settings:

```
@Id
@GeneratedValue(generator = "table", strategy=GenerationType.TABLE)
@TableGenerator(name = "table", allocationSize = 3)
private Long id;
```

If the `hibernate.id.new_generator_mappings` configuration property is false, then Hibernate will choose the `MultipleHiLoPerTableGenerator`. This generator requires a single table for managing multiple identifiers, and just like `SequenceHiLoGenerator`, it also uses the hi/lo algorithm by default.

When the enhanced identifier generators are activated, Hibernate uses the `TableGenerator` instead, which can also take configurable optimizer strategies.

For both the enhanced sequence and the table identifier generator, Hibernate comes with the following built-in optimizers:

Table 9.8: Hibernate identifier optimizers

Optimizer type	Implementation class	Description
`none`	`NoopOptimizer`	Every identifier is fetched using a new roundtrip to the database
`hi/lo`	`HiLoOptimizer`	It allocates identifiers by using the legacy hi/lo algorithm
`pooled`	`PooledOptimizer`	It is an enhanced version of the hi/lo algorithm which is interoperable with other systems unaware of this identifier generator
`pooled-lo`	`PooledLoOptimizer`	It is a variation of the `pooled` optimizer, the database sequence value representing the *lo* value instead of the *hi* one

By default, the `SequenceStyleGenerator` and `TableGenerator` identifier generators uses the `pooled` optimizer. If the `hibernate.id.optimizer.pooled.prefer_lo` configuration property is set to true, Hibernate will use the `pooled-lo` optimizer by default.

Both the `pooled` and the `pooled-lo` encode the database sequence value into the identifier range boundaries, so allocating a new value using the actual database sequence call does not interfere with the identifier generator allocation process.

9.2.3.4.4 The pooled optimizer

The pooled optimizer can be configured as follows:

```
@Entity
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "pooled")
    @GenericGenerator(
        name = "pooled",
        strategy = "org.hibernate.id.enhanced.SequenceStyleGenerator",
        parameters = {
            @Parameter(name = "sequence_name", value = "sequence"),
            @Parameter(name = "initial_value", value = "1"),
            @Parameter(name = "increment_size", value = "3"),
            @Parameter(name = "optimizer", value = "pooled")
        }
    )
    private Long id;
}
```

The increment size gives the range of values allocated by the sequence generator with one database roundtrip. Although it is inefficient to flush the Persistence Context after every `persist` method call, in this test, the flush outlines when the database sequence was called.

```
doInJPA(entityManager -> {
    for (int i = 0; i < 5; i++) {
        entityManager.persist(new Post());
        entityManager.flush();
    }
    entityManager.unwrap(Session.class).doWork(connection -> {
        try(Statement statement = connection.createStatement()) {
            statement.executeUpdate(
                "INSERT INTO post VALUES NEXT VALUE FOR sequence"
            );
        }
    });
    for (int i = 0; i < 3; i++) {
        entityManager.persist(new Post());
        entityManager.flush();
    }
});
```